

Documentación Fase 2 — useContext · useRef · Tema Visual

Descripción general

En esta fase se implementó una arquitectura basada en Context API para separar la lógica de almacenamiento y tema visual de los componentes principales de la aplicación.

También se agregaron funcionalidades usando useRef y atajos de teclado.

La aplicación ahora puede trabajar en dos modos:

- LocalStorage
- API Backend

Además, se implementó un sistema de tema claro/oscuro persistente y categorías centralizadas para las sesiones de entrenamiento.

StorageContext

Se creó un contexto llamado StorageProvider.jsx para manejar toda la lógica de almacenamiento de la aplicación.

Este contexto expone:

- modo
- setModo
- obtenerItems()
- guardarItem()
- eliminarItem()
- editarItem()

El objetivo fue que los componentes no supieran si los datos vienen desde LocalStorage o desde la API.

Modo híbrido Local/API

Se agregó un sistema para cambiar dinámicamente entre:

- modo local
- modo API

Cuando el usuario cambia el modo desde la interfaz:

- los datos se actualizan automáticamente
- el modo se guarda en localStorage
- los componentes siguen funcionando sin cambios

Esto permitió mantener una sola interfaz para ambos tipos de almacenamiento.

ThemeContext

Se implementó un segundo contexto llamado ThemeProvider.jsx.

Este contexto controla:

- tema claro
- tema oscuro
- persistencia del tema

El tema se guarda automáticamente en localStorage para mantener la preferencia del usuario después de recargar la página.

Variables CSS

Se reemplazaron varios colores hardcodeados por variables CSS usando:

body[data-theme="claro"]

body[data-theme="oscuro"]

Esto permitió cambiar completamente la apariencia visual sin modificar componentes individuales.

useRef

Se implementaron dos usos distintos de useRef.

1. Focus automático en input

Después de crear una sesión, el cursor vuelve automáticamente al input principal para facilitar el ingreso rápido de nuevas sesiones.

También se agregó el atajo:

alt + N

para enfocar manualmente el input desde cualquier parte de la aplicación.

2. Referencia de setInterval

Se utilizó useRef para guardar el identificador de un setInterval sin provocar re-renderizados innecesarios.

También se agregó cleanup usando:

`clearInterval()`

Atajos de teclado

Se implementaron dos atajos:

Atajo	Acción
alt + N	Enfocar input principal
T	Cambiar tema

Los eventos fueron registrados usando `addEventListener` y eliminados correctamente usando `removeEventListener` **dentro del cleanup de useEffect.**

Categorías

Se creó el archivo:

`src/utils/categorias.js`

Las categorías incluyen:

- id
- nombre
- color
- emoji vacío

Las categorías actuales son:

- Fuerza
- Cardio

- Flexibilidad
- Deportes
- Movilidad

Vista calendario

La vista de calendario fue agregada como una funcionalidad adicional y no como un requisito obligatorio de la fase.

La idea principal de esta sección es comenzar a estructurar una futura visualización temporal de sesiones y actividades físicas, permitiendo observar entrenamientos organizados por fecha.

Actualmente la vista funciona como una representación básica de los días con actividad registrada, pero se planea expandirla más adelante con:

- estadísticas por fecha
- gráficas de progreso
- seguimiento semanal y mensual
- visualización de tendencias
- análisis de actividad física

Esta sección tendrá mayor profundidad y utilidad durante la Fase 3, cuando se implemente el sistema de gráficas dinámicas usando Recharts y useReducer.

Mejoras visuales

Durante esta fase también se mejoraron varios aspectos visuales:

- filtros activos resaltados
- edición de sesiones más limpia
- modo claro/oscuro
- sincronización automática entre API y LocalStorage
- mejor organización visual de las cards